

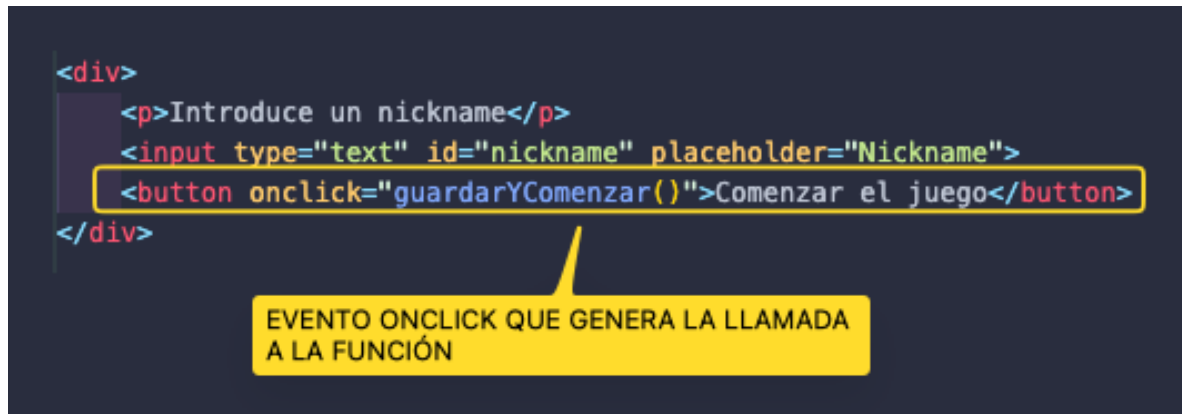
DOCUMENTACIÓN PÁGINA WEB CUANTOSABES

GESTIÓN DEL NICKNAME

Actualmente nuestra página inicial requiere del ingreso de un nickname (nombre de usuario) para comenzar a jugar.

Para ello, hemos previsto que el botón comenzar juego dispare un evento (onclick) que genera una llamada a la función llamada guardarYComenzar().

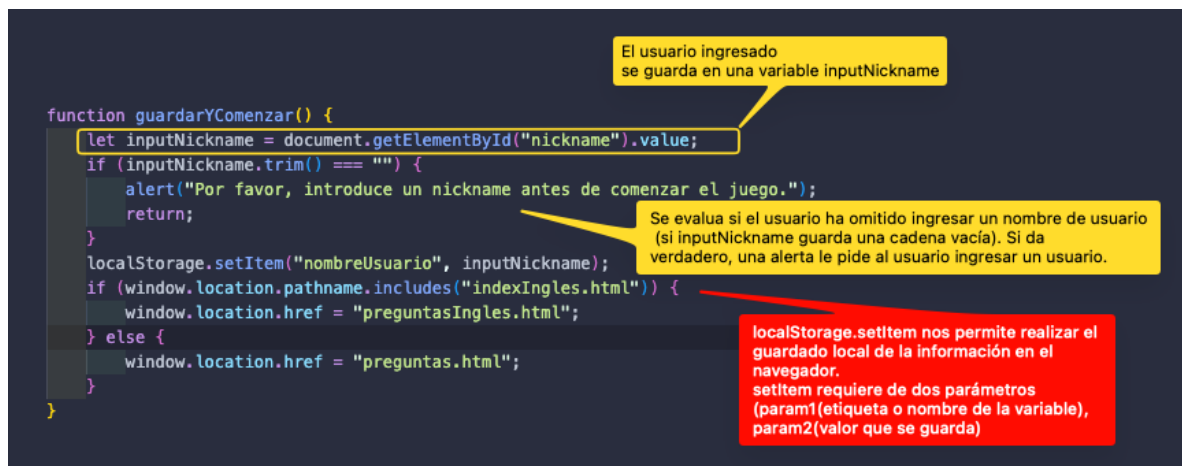
INDEX.HTML



SCRIPT.JS

Como necesitamos guardar un valor desde una página para pasarlo a otra página diferente, necesitamos utilizar localStorage.

Esto es la memoria del propio navegador web que permite guardar información del sitio en variables con una etiqueta, a la cual luego podemos acceder:



Como ya tenemos el nombre de usuario que necesitamos, tenemos que hacer el redireccionamiento a la página de preguntas.

Sin embargo, nuestro sitio web tiene dos opciones de idiomas (Español-Inglés) por lo que para verificar a cual página enlazar (preguntas.html o preguntasIngles.html) utilizamos la siguiente línea:

```
if (window.location.pathname.includes("indexIngles.html")) {  
    window.location.href = "preguntasIngles.html";  
} else {  
    window.location.href = "preguntas.html";  
}  
}
```

Entonces utilizamos el objeto **window** en JS que representa la ventana actual de nuestro navegador.

Al utilizar window.location podemos extraer o modificar diferentes piezas de información de la URL.

Con **window.location.pathname.includes("indexIngles.html")** el script analiza la dirección URL del navegador para ver si encuentra en alguna parte de la URL "indexIngles.html" lo cual nos diría que estamos en el sitio web en inglés.

Si la condición arroja respuesta verdadera, utilizamos window.location.href que, al añadirle un igual + el nombre de un archivo, le damos orden directa al navegador de que cambie de página como si hicieramos click en una etiqueta <a> de HTML.

Es así como determinamos a que archivo hacer el direccionamiento, dependiendo del idioma en el que está el sitio web en el que navega el usuario.

MOSTRANDO USUARIO ACTIVO Y CARGANDO PREGUNTAS

Para mostrar el usuario activo y cargar las primeras preguntas del juego hemos utilizado las siguientes líneas en JS:

```
document.addEventListener("DOMContentLoaded", () => {
    let elementoJugador = document.getElementById("display-jugador");

    // Verificamos si el elemento existe (para que no de error en el index)
    if (elementoJugador) {
        // Recuperamos el nombre guardado
        let nombreGuardado = localStorage.getItem("nombreUsuario");

        if (nombreGuardado) {
            elementoJugador.textContent = nombreGuardado;
        }
        cargarPreguntas(); // Llamamos a la función para cargar las preguntas
    }
});
```

Desglose de la primera línea para entenderla mejor:

- document. → Document es un objeto en JS que representa TODO el archivo HTML.
- addEventListener → Actúa como un “escuchador de eventos”.
Decimos entonces al navegador, que activaremos un escuchador de eventos que ocurran en ese documento html.
Sin embargo, para completar la acción debemos establecer:
 - Que evento debe escuchar
 - Que acción debe realizar cuando ese evento ocurra
- ("DOMContentLoaded ... → Es el evento que desencadena la ejecución de la acción.
DOMContentLoaded (El Modelo de Objetos del Documento HTML) consiste en esperar que el navegador haya leído todo el HTML de arriba abajo, sin esperar que carguen recursos pesados como imágenes o el CSS.
Esto es clave ya que si JS intenta buscar, por ejemplo, la caja display-jugador una milésima de segundo antes de que el navegador haya procesado esa línea del HTML, dará un error diciendo que el elemento no existe.
- Función flecha: () → { ... } → Es la orden a ejecutar al dispararse el evento.

Desglose de la función fecha:

- () los paréntesis se utilizan para recibir datos o parámetros, que en este caso están vacíos por que no le pasamos nada.
- → la flecha conecta los paréntesis con el código, diciendo ENTONCES ejecuta esto
- {...} envuelven todo el bloque de código que se ejecutará cuando se dispare el evento del addEventListener.

Desglose del bloque de acciones a ejecutar:

```
let elementoJugador = document.getElementById("display-jugador");  
  
// Verificamos si el elemento existe (para que no de error en el index)  
if (elementoJugador) {  
    // Recuperamos el nombre guardado  
    let nombreGuardado = localStorage.getItem("nombreUsuario");  
  
    if (nombreGuardado) {  
        elementoJugador.textContent = nombreGuardado;  
    }  
    cargarPreguntas(); // Llamamos a la función para cargar las preguntas  
}};
```

- 1) Nos guardamos el elemento HTML que debe mostrar el nombre de usuario en una variable elementoJugador
- 2) Antes de intentar rellenar ese elemento, establecemos un método de verificación de que ese elemento exista a través de un if(elementoJugador). Solo si este condicional arroja verdadero (lo cual implica que el elemento existe), se crea una variable que guardará la información almacenada en el localStorage.

Veamos como cambia la sintaxis respecto al localStorage:

GUARDADO DE INFORMACIÓN

```
localStorage.setItem("nombreUsuario", inputNickname);
```

setItem

+

"nombreDeLaEtiqueta"

+

valor

ACCESO A INFORMACION GUARDADA

```
localStorage.getItem("nombreUsuario");
```

getItem

+

"nombreDeLaEtiqueta"

Luego realizamos otra verificación de que efectivamente existe contenido mediante un if de verificación, y si la condición es verdadera se le agrega al elemento HTML el nombre guardado a través de `.textContent=nombreGuardado`

BUSQUEDA DE LAS PREGUNTAS EN ARCHIVO .JSON Y SU PREPARACIÓN PARA EL JUEGO

```
// Carga de las preguntas a partir de un archivo JSON externo
let listaPreguntas = [];

async function cargarPreguntas() {
  try {
    let archivo;
    if (window.location.pathname.includes("preguntasIngles.html")) {
      archivo = "preguntasIngles.json";
    } else {
      archivo = "preguntas.json";
    }

    const respuesta = await fetch(archivo);
    const datos = await respuesta.json();

    listaPreguntas = datos;

    prepararJuego(); // Llamamos a la función para preparar el juego con las preguntas cargadas
  } catch (error) {
    // Gestión de errores: Si el archivo no existe o falla la red, entra aquí
    console.error("Hubo un problema al cargar el JSON:", error);
  }
}
```

En primer lugar, hemos creado la variable `listaPreguntas=[]` que recibirá el array de objetos que le entregue el JSON como una variable global, por fuera de la función.

Esto es útil ya que debemos utilizarla en otras partes del código, por fuera de la función.

Ahora sí, analicemos en profundidad la función:

➤ `async function cargarPreguntas()`

Hemos añadido el `async` al inicio de la declaración de la función. Esto advierte a JS que es una función que puede tomar tiempo ya que depende de archivos externos para que no congele la página mientras espera.

➤ Bloque de código a ejecutar: utilizamos `try catch`.

Como dependemos de un archivo externo que puede generar errores, lo mejor es gestionarlos para que la página no colapse.

Por eso envolvemos todo lo que debería hacer la función en el try, y prevemos el catch para que en caso de que se de un error, se capture y se muestre en consola en lugar de colapsar la página.

ANALISIS DEL TRY

```
let archivo;  
if (window.location.pathname.includes("preguntasIngles.html")) {  
  archivo = "preguntasIngles.json";  
} else {  
  archivo = "preguntas.json";  
}
```

Salimos a buscar el archivo externo .json que contiene las preguntas. Pero como el sitio web tiene la opción de inglés, retomamos el método que ya utilizamos para determinar en que versión del sitio estamos y descargar así el .json correcto.

```
const respuesta = await fetch(archivo);  
  
const datos = await respuesta.json();  
  
listaPreguntas = datos;  
  
prepararJuego(); // Llamamos a la función para preparar el juego con las
```

- **Fetch** → Mensajero que busca el archivo
- **Await** → Como JS tiene que buscar, abrir y desestructurar todo un archivo externo, esto puede llevar tiempo por lo que await PAUSA LA EJECUCIÓN DE LA FUNCIÓN EN ESE PUNTO hasta que el fetch (el mensajero) vuelve con el archivo descargado
- **Respuesta.json** → El fetch al regrear no trae el archivo ya listo para usarse, sino que trae un paquete con datos en bruto. Al agregar .json desempaquetamos esos datos y lo convertimos en un array de objetos legible por JS. Este desempaquetado puede llevar tiempo, por ello, volvemos a utilizar await para pausar la ejecución de la función.

- `listaPreguntas=datos` → Si no tuviéramos que usar el contenido de ese archivo fuera de la función ya nos valdría usar la variable `datos`, pero como necesitamos ese array de objetos fuera de la función, lo asignamos a `listaPreguntas` que es la variable global que ya hemos creado.
- Al final, llamamos a la siguiente función (`prepararJuego`) para empezar a mostrar por pantalla las preguntas

ANÁLISIS DEL CATCH

```
} catch (error) {  
  console.error("Hubo un problema al cargar el JSON:", error);  
}  
}
```

NOTA → `console.error` imprime la salida del mensaje en rojo y con el logo de advertencia.

PREPARACIÓN DE LAS PREGUNTAS

BARAJANDO LAS PREGUNTAS

Es necesario que de las 20 preguntas que tenemos se muestren solo 4, pero no tienen que ser siempre las primeras cuatro.

Es por ello que necesitamos mezclar las preguntas almacenadas en el `.JSON`, es decir, mezclar el array.

```
function barajarArray(array) {  
  for (let i = array.length - 1; i > 0; i--) {  
    let j = Math.floor(Math.random() * (i + 1));  
    // Intercambiamos los elementos en las posiciones i y j  
    let temporal = array[i];  
    array[i] = array[j];  
    array[j] = temporal;  
  }  
  return array;  
}
```

Analicemos la estructura de la función parte por parte. Lo que necesitaremos es realizar un cambio de posiciones de las funciones.

Para ello necesitamos → Primer elemento del array que vamos a intercambiar

```
for (let i = array.length - 1; i > 0; i--) {
```

El índice *i* almacenará la cantidad de elementos que tiene el array -1. Esto nos dará como resultado la última posición del array que es donde se almacena el último elemento.

Al final de la vuelta, el valor de índice *i* se decrementa en uno

Luego necesitamos determinar por cuál elemento del array vamos a realizar el intercambio

```
let j = Math.floor(Math.random() * (i + 1));
```

- `Math.random()` → Genera un número entre 0 y 0.99, nunca llega a uno. Como nunca suma uno, si solo lo multiplicamos por el valor actual del índice, nunca llegaríamos a contemplar el último elemento del array

Imaginemos que el array tiene tres elementos, por lo que sus posiciones van desde 0 a 2.

`Math.random()` nos da 0.85, multiplicado por 2 nos daría 1.998. Al hacer `Math.floor` que recorta los decimales, nos quedaría en 1 por lo que la última posición siempre queda fuera del sorteo.

Es por ello que se multiplica por $(i+1)$. Estiramos el limite superior para que siempre contemple el último lugar.

Una vez definidos los dos elementos, es decir, el que vamos a cambiar de posición y el que lo reemplazará, creamos una variable temporal que almacene el valor original para no perderlo en el camino:

```
let temporal = array[i];  
array[i] = array[j];  
array[j] = temporal;
```

1) Temporal almacena el valor de la posición original

2) El valor de la posición original es reemplazado por el elemento surgido del azar

3) El valor del elemento surgido por el azar es reemplazado por el de la posición original

APLICACIÓN DEL BARAJE Y PREPARACIÓN DEL JUEGO

```
let preguntasJuego = [];
```

Creamos un nuevo array como variable global (ya que necesitaremos usarla por fuera de esa función). Allí guardaremos el array resultante de quedarnos únicamente con las primeras cuatro preguntas.

```
function prepararJuego() {  
  barajarArray(listaPreguntas);  
  preguntasJuego = listaPreguntas.slice(0, 4);  
}
```

Utilizamos la función creada anteriormente para realizar el baraje del array y mezclar las preguntas.

Una vez mezcladas, guardamos en el nuevo array el resultado de cortar (slice desde la pregunta de la posición 0 a la posición 4 (esta última no se incluye) el array original

Como el array de preguntas guarda la respuesta correcta como índice, al mezclar las respuestas para que aparezcan en un orden distinto en cada vuelta cambiarían sus índices y la correcta dejaría de coincidir.

Por eso, ANTES de realizar el “baraje” de respuestas, nos guardamos el texto almacenado en el índice correcto.

```
preguntasJuego.forEach(pregunta => {  
    pregunta.textoRespuestaCorrecta = pregunta.respuestas[pregunta.correcta];  
    barajarArray(pregunta.respuestas);  
});  
dibujarRonda(); // Llamamos a la función
```

En cada pregunta, busca entre las respuestas el texto que se encuentra en la posición que almacena correcta (se queda con ese índice) y lo va a almacenar en la nueva propiedad textoRespuestaCorrecta

Entramos al nuevo array que solo tiene las cuatro preguntas y lo recorremos con un forEach.

(pregunta → Es un índice que empieza en la posición 0 y llega hasta la última posición)

En cada vuelta, añade una nueva propiedad al array llamada textoRespuestaCorrecta.

Una vez guardado el texto de la respuesta correcta en la nueva propiedad del array, volvemos a utilizar la función del baraje, pero pasándole ahora el array de respuestas.

Cuando ha finalizado el forEach, al salir, llamamos a la función dibujarRonda() que nos ayudará a imprimir en la pantalla las preguntas del juego.

DIBUJANDO LAS PREGUNTAS EN PANTALLA

Partimos de la base de que hemos creado en nuestro archivo preguntas.html una caja en la que iremos inyectando código html a partir de JS

```
<main class="contenedor-juego">  
    <div id="contenedor-preguntas"></div>  
    <button id="btn-comprobar" class="btn-neon" onclick="comprobarRespuestas()">Comprobar respuestas</button>  
</main>
```

```

let rondaActual = 1;

function dibujarRonda() {
  let contenedor = document.getElementById("contenedor-preguntas");
  contenedor.innerHTML = "";

  // IMPORTANTE: Ahora dibujamos desde la primera pregunta (0) hasta el límite de la ronda actual
  let fin = rondaActual * 2;
  let preguntasMostrar = preguntasJuego.slice(0, fin);

  let html = "";

  preguntasMostrar.forEach((pregunta, index) => {
    // Si el índice de la pregunta es menor a las de la ronda actual, significa que es antigua y la bloqueamos
    let bloqueado = (index < (rondaActual - 1) * 2) ? "disabled" : "";

    html += `<div class="tarjeta-pregunta">
      <h4>Pregunta ${index + 1}</h4>
      <h3>${pregunta.pregunta}</h3>
      <div class="opciones">`;

    pregunta.respuestas.forEach((respuesta) => {
      // Si el usuario ya habia elegido esta opción en una ronda anterior, le ponemos "checked" para no borrar su selección
      let marcado = (pregunta.respuestaUsuario === respuesta) ? "checked" : "";

      html += `<label>
        <input type="radio" name="pregunta-${pregunta.id}" value="${respuesta}" ${bloqueado} ${marcado}>
        ${respuesta}
      </label><br><br>`;
    });

    html += `</div></div><hr>`;
  });

  contenedor.innerHTML = html;

  // Detalle visual extra: Cambiamos el texto del botón en la última ronda
  let btnComprobar = document.getElementById("btn-comprobar");
  if (btnComprobar && rondaActual === 2) {
    btnComprobar.textContent = "Comprobar y Finalizar";
  }
}

```

- Creamos una variable `rondaActual=1` que es global, por fuera de la función. Esto es porque luego necesitamos que cambie de valor y vuelva a ejecutar el interior de `dibujarRonda()` pero con un valor que se ha modificado externamente a esa función.

```
function dibujarRonda() {
  let contenedor = document.getElementById("contenedor-preguntas");
  contenedor.innerHTML = "";
```

contenedor guarda el elemento HTML donde inyectará las preguntas en forma de tarjetas.

Esto es muy importante, porque se asegura de limpiar la etiqueta en cada vuelta.

Como en la primer ejecución de la función, pintará las primeras dos preguntas, y en la segunda ejecución pintará las cuatro, es necesario que al pintar las 4 esté vacío.

Por ello al principio de la ejecución de la función, limpia la etiqueta.

```
let fin = rondaActual * 2;
let preguntasMostrar = preguntasJuego.slice(0, fin);
```

```
let html = "";
```

En esta variable vamos a ir almacenando paso por paso lo que inyectaremos de una sola vez en contenedor.

Aquí determinamos cuantas preguntas mostrar.

Como rondaActual=1, fin vale 2.

Se crea un nuevo array preguntasMostrar con un recorte de preguntasJuego (slice) que va desde la pregunta con índice 0 hasta la pregunta con índice 2, sin mostrar esta última.

Es decir, nos quedamos con los índices 0 y 1.

EN ESTAS LÍNEAS DETERMINAMOS DOS COSAS: SI LA PREGUNTA SE IMPRIME BLOQUEADA, Y SI EL RADIO BUTTON SE IMPRIME MARCADO O NO.

```
preguntasMostrar.forEach((pregunta, index) => {
  // Si el índice de la pregunta es menor a las de la ronda actual, signif.
  let bloqueado = (index < (rondaActual - 1) * 2) ? "disabled" : "";
```

Creamos un forEach que recorrerá cada pregunta del array preguntasMostrar, teniendo en cuenta también su índice (index)

Esta línea determina el bloqueo de las preguntas en la pantalla.

En la primer ronda no se bloquearan, pero en la segunda necesitamos que las primeras dos queden bloqueadas.

```
html += `<div class="tarjeta-pregunta">
  <h4>Pregunta ${index + 1}</h4>
  <h3>${pregunta.pregunta}</h3>
  <div class="opciones">`;
```

Vamos almacenando en la etiqueta que se inyectará, la información que debe imprimirse

Esta última captura requiere un poco más de explicación:

En esta primera ronda, preguntasMostrar solo guarda dos preguntas, con índice (index) 0 y 1.

Además, rondaActual es igual a 1.

Utiliza un ternario para determinar el valor de bloqueado.

```
let bloqueado = (index < (rondaActual - 1) * 2) ? "disabled" : "";
```

index=0 es menor que 0 ? → FALSO bloqueado=""
index=1 es menor que 0 ? → FALSO bloqueado=""

Pero cuando rondaActual=2 y preguntasMostrar tenga las cuatro preguntas ocurrirá lo siguiente:

index=0 es menor que 2 ? → VERDADERO bloqueado="disabled"
index=1 es menor que 2 ? → VERDADERO bloqueado="disabled"
index=2 es menor que 2 ? → FALSO bloqueado=""
index=3 es menor que 2 ? → FALSO bloqueado=""

Recorremos las respuestas con un forEach. En cada vuelta determinamos SI EL RADIO (CIRCULO) HA DE IMPRIMIRSE MARCADO Y BLOQUEADO O NO.

```
pregunta.respuestas.forEach((respuesta) => {  
  // Si el usuario va habia elegido esta opción en una ronda anterior, le ponemos "checked" para no borrar su selección  
  let marcado = (pregunta.respuestaUsuario === respuesta) ? "checked" : "";  
  
  html += `<label>  
    <input type="radio" name="pregunta-${pregunta.id}" value="${respuesta}" ${bloqueado} ${marcado}>  
    ${respuesta}  
  </label><br><br>`;  
});
```

Añade una propiedad al array respuestaUsuario y utiliza un ternario para determinar el valor de marcado

En la primera vuelta (rondaActual=1) siempre el radio estará en blanco "" y el bloqueado desactivado.

¿Por qué imprimirá el radio en blanco?

Porque la propiedad respuestaUsuario se crea vacía, entonces la comparación con respuesta siempre será falsa.

DETALLE VISUAL → Cambiando el texto del boton de comprobar para que sea intuitiva la finalización del juego:

```
// Detalle visual extra: Cambiamos el texto del botón en la última ronda
let btnComprobar = document.getElementById("btn-comprobar");
if (btnComprobar && rondaActual === 2) {
    btnComprobar.textContent = "Comprobar y Finalizar";
}
}
```

Se guarda en la variable btnComprobar la etiqueta con ese ID

Evalúa una condición doble:

- btnComprobar existe?
- rondaActual === 2?

Si ambas dan verdadera, modifica el texto de la etiqueta btnComprobar por Comprobar y Finalizar. Si es rondaActual1 no hará nada.

EVALUACIÓN DE LAS RESPUESTAS CORRECTAS:

```

// 1. Definir función para comprobar
function comprobarRespuestas() {
    // Calculamos qué preguntas estamos evaluando en ESTA ronda
    let inicio = (rondaActual - 1) * 2;
    let fin = inicio + 2;
    let preguntasRonda = preguntasJuego.slice(inicio, fin);

    let todasCorrectas = true;
    let faltanRespuestas = false;

    preguntasRonda.forEach(pregunta => {
        // Magia de JS: Buscamos el radio button que el usuario haya marcado para el ID de esta pregunta en concreto
        let seleccion = document.querySelector(`input[name="pregunta-${pregunta.id}"]:checked`);

        if (!seleccion) {
            faltanRespuestas = true; // El jugador no marcó nada en esta pregunta
        } else {
            // Guardamos la respuesta del usuario en nuestro objeto para no perderla cuando redibujemos el HTML
            pregunta.respuestaUsuario = seleccion.value;

            // Comparamos el texto del radio button con el que guardamos al principio
            if (seleccion.value !== pregunta.textoRespuestaCorrecta) {
                todasCorrectas = false; // El jugador falló
            }
        }
    });

    // 1. Validar que no se dejó nada en blanco
    if (faltanRespuestas) {
        alert("¡Eh! Te has dejado alguna pregunta sin responder en esta ronda.");
        return;
    }

    // 2. Comprobar si ganó o perdió
    if (!todasCorrectas) {
        // Si hay algún fallo, termina el juego enviando "false"
        finalizarJuego(false);
    } else {
        if (rondaActual === 1) {
            alert("¡Correcto! Pasas a la ronda final.");
            rondaActual++;
            dibujarRonda(); // Redibujamos la pantalla
        } else {
            // Si es la ronda 2 y acertó todo, termina el juego enviando "true"
            finalizarJuego(true);
        }
    }
}

```

Analizamos paso a paso:

DETERMINACIÓN DE LAS PREGUNTAS A EVALUAR EN ESTA RONDA

// NUEVA FUNCIÓN PARA EVALUAR

```
function comprobarRespuestas() {
```

```
  let inicio = (rondaActual - 1) * 2;
```

```
  let fin = inicio + 2;
```

```
  let preguntasRonda = preguntasJuego.slice(inicio, fin);
```

Se crea un array
con esa porción de preguntas

Si rondaActual=1

inicio = 0

fin = 2

Se quedará solo con las preguntas con índice 0 y 1

Si rondaActual=2

inicio=2

fin=4

Se quedará solo con las preguntas con índice 2 y 3.

BANDERAS CLAVE

```
let todasCorrectas = true;
```

```
let faltanRespuestas = false;
```

SE SUPONEN DOS COSAS:

- todasCorrectas → Que el usuario ha contestado todas las preguntas bien

-faltanRespuestas → Que el usuario ha contestado todas las preguntas

Estas banderas podrán cambiar de valor en las evaluaciones.

LA EVALUACIÓN:

```
preguntasRonda.forEach(pregunta => {
  let seleccion = document.querySelector(`input[name="pregunta-${pregunta.id}"]:checked`);

  if (!seleccion) {
    faltanRespuestas = true;
  } else {
    // Guardamos la respuesta del usuario en nuestro objeto para no perderla cuando redibujemos el HTML
    pregunta.respuestaUsuario = seleccion.value;

    // Comparamos el texto del radio button con el que guardamos al principio
    if (seleccion.value !== pregunta.textoRespuestaCorrecta) {
      todasCorrectas = false; // El jugador falló
    }
  }
});
```

Vamos por partes:

SELECCIÓN GUARDARÁ EL ELEMENTO HTML QUE SE HAYA MARCADO

```
preguntasRonda.forEach(pregunta => {
  let seleccion = document.querySelector(`input[name="pregunta-${pregunta.id}"]:checked`);
```

Buscamos en el grupo de respuestas posibles para el id de nuestra pregunta, aquel que tenga la propiedad checked.

Esta propiedad es interna del DOM y se activa (se pone true) cuando se selecciona una opción (click en el radio).

```
<label>
  <input type="radio" name="pregunta-9" value="Español">
    " Español "
</label>
<br>
<br>
<label>
  <input type="radio" name="pregunta-9" value="Hindi">
    " Hindi "
</label>
<br>
<br>
<label>
  <input type="radio" name="pregunta-9" value="Inglés">
    " Inglés "
</label>
<br>
<br>
<label> == $0
  <input type="radio" name="pregunta-9" value="Chino Mandarín">
    " Chino Mandarín "
```

SELECCIÓN TENDRÁ DOS VALORES
POSIBLES:

- EL ELEMENTO QUE CUYA PROPIEDAD ES CHECKED
- NULL (SI NO HA HABIDO NINGUN CHECKED, SE QUEDA VACÍO)

```
if (!seleccion) {  
    faltanRespuestas = true;  
}
```

Si seleccion contiene un checked, es TRUE y su opuesto siempre es FALSE = No entra al if

Si selección es NULL, es FALSE, su opuesto es TRUE por lo que entraría al IF y cambiaría el estado de la bandera faltanRespuestas

Si no ha entrado al if porque hay selección, ejecuta el else:

```
} else {  
    // Guardamos la respuesta del usuario en nuestro  
    pregunta.respuestaUsuario = seleccion.value;  
}
```

Almacena en la propiedad respuestaUsuario que habíamos creado anteriormente, el valor que almacena el elemento checked.

Hasta aquí ha corroborado si se han contestado ambas preguntas, ahora validará si son correctas o no:

```
// Comparamos el texto del radio button con el que guardamos  
if (seleccion.value !== pregunta.textoRespuestaCorrecta) {  
    todasCorrectas = false; // El jugador falló  
}  
}
```

Comparamos si los textos coinciden.

En caso de no haber coincidencia se cambia el valor de la bandera todasCorrectas a falso.

Sino, se mantendrá verdadero.

Como ya tenemos definidos los valores reales de nuestras banderas, veamos como procede a ejecutar acciones en base a esas banderas:

```
if (faltanRespuestas) {  
    alert("¡Eh! Te has dejado alguna pregunta sin responder en esta ronda.");  
    return;  
}
```

Si faltanRespuestas ha cambiado a True, abrirá una ventana con un mensaje y gracias al return, para la ejecución de la función con lo cual no pasará a evaluar si son correctas o no.

En el caso de que faltanRespuestas haya quedado en False, ese condicional no se ejecuta y pasa directamente evaluar el siguiente:

```
// 2. Comprobar si ganó o perdió  
if (!todasCorrectas) {  
    // Si hay algún fallo, termina el juego  
    finalizarJuego(false);  
} else {  
    if (rondaActual === 1) {  
        alert("¡Correcto! Pasas a la ronda final.");  
        rondaActual++;  
        dibujarRonda(); // Redibujamos la pantalla  
    } else {  
        // Si es la ronda 2 y acertó todo, termina el juego enviando "true"  
        finalizarJuego(true);  
    }  
}
```

Solo entra si todasCorrectas es FALSE y realiza la llamada a la función finalizarJuego pasandole como parámetro el valor FALSE

DETERMINACIÓN IMPORTANTE: EL JUEGO CONTINUA O FINALIZA

Si todas correctas ha sido verdadero, revisa en que ronda se encuentra. Si la rondaActual es 1, comunica el pase a la siguiente ronda con una ventana, realiza el incremento de rondaActual (valdrá 2) y vuelve a llamar dibujarRonda() para que se pinten las cuatro preguntas, con las dos primeras ahora como bloqueadas
SOLO ENTRA A ESTE IF SI RONDA ACTUAL ES 1

Si todas correctas ha sido verdadero y ronda actual no es 1, no entra a ese if y ejecuta el else: llamar a finalizarJuego con el parámetro en TRUE

RESULTADO FINAL DEL JUEGO:

```
function finalizarJuego(haGanado) {  
  let acertadas = 0;  
  let falladas = 0;  
  
  // 1. Contamos aciertos y fallos  
  preguntasJuego.forEach(pregunta => {  
    if (pregunta.respuestaUsuario) {  
      if (pregunta.respuestaUsuario === pregunta.textoRespuestaCorrecta) {  
        acertadas++;  
      } else {  
        falladas++;  
      }  
    }  
  });  
  
  let nombreJugador = localStorage.getItem("nombreUsuario") || "Invitado";  
  
  // 2. Creamos el objeto JSON tal y como lo pide el enunciado  
  const datosFinales = {  
    jugador: nombreJugador,  
    acertadas: acertadas,  
    falladas: falladas,  
    victoria: haGanado  
  };  
  
  // Imprimimos el JSON por consola para demostrar que está empaquetado y listo  
  console.log("JSON listo (simulación de envío):", JSON.stringify(datosFinales));  
  
  // 3. Modificamos el contenido del Modal  
  let modal = document.getElementById("modal-resultados");  
  let mensajeFinal = document.getElementById("mensaje-final");  
  let puntuacionFinal = document.getElementById("puntuacion-final");  
  
  if (haGanado) {  
    mensajeFinal.textContent = `¡Felicidades, ${nombreJugador}! ¡Has ganado el concurso!`;  
  } else {  
    mensajeFinal.textContent = `¡Fin del juego, ${nombreJugador}!`;  
  }  
  
  // Mostramos la puntuación en formato X / Y  
  puntuacionFinal.textContent = `${acertadas} / ${acertadas + falladas} Correctas`;  
  
  // 4. Hacemos visible la ventana emergente quitándole la clase "oculto"  
  modal.classList.remove("oculto");  
}
```

Análisis en detalle:

```
function finalizarJuego(haGanado) {  
  let acertadas = 0;  
  let falladas = 0;  
  
  // 1. Contamos aciertos y fallos  
  preguntasJuego.forEach(pregunta => {  
    if (pregunta.respuestaUsuario) {  
      if (pregunta.respuestaUsuario === pregunta.textoRespuestaCorrecta) {  
        acertadas++;  
      } else {  
        falladas++;  
      }  
    }  
  });  
  
  let nombreJugador = localStorage.getItem("nombreUsuario") || "Invitado";  
}
```

Creamos una función que ha de recibir un valor.

Recorremos nuestro array de las 4 preguntas comparando dos propiedades:
- ¿respuestaUsuario es igual a textoRespuestaCorrecta?
El resultado incrementará los contadores

Recuperamos el nombreJugador del localStorage para preparar el .JSON para enviarle al servidor los resultados

```
// 2. Creamos el objeto JSON tal y como lo pide el enunciado  
const datosFinales = {  
  jugador: nombreJugador,  
  acertadas: acertadas,  
  falladas: falladas,  
  victoria: haGanado  
};  
  
// Imprimimos el JSON por consola para demostrar que está empaquetado y listo  
console.log("JSON listo (simulación de envío):", JSON.stringify(datosFinales));
```

Creamos un objeto con las propiedades que solicitaba la práctica en el punto 7 a los efectos de que luego se convierta en un .json para el servidor

Esto es clave: Así como antes convertimos el texto plano de las preguntas que recibíamos del .json en un objeto JS, hacemos lo opuesto.

Con `JSON.stringify(datosFinales)` convertimos el objeto JS en texto plano para poder enviárselo al servidor

NOTA: Se añade un `console.log` para poder ver en consola su funcionamiento.

RESULTADOS FINALES EN PANTALLA → EL TRUCO DEL MODAL

Para cambiar la visual que entregan los alert, utilizamos un modal para mostrar el resultado final del juego.

```
<div id="modal-resultados" class="oculto">
  <div class="contenido-modal">
    <h2 id="mensaje-final"></h2>
    <p class="texto-gris">PUNTUACIÓN FINAL</p>
    <h3 id="puntuacion-final" class="texto-amarillo"></h3>
    <button class="btn-neon" onclick="window.location.href='index.html'">Volver a jugar</button>
  </div>
</div>
```

Como podemos ver en nuestro HTML existe de forma permanente, pero no se muestra porque tiene la clase "oculto"

A su vez, vemos que está preparado para recibir información ya que contiene etiquetas vacías.

```
// 3. Modificamos el contenido del Modal
let modal = document.getElementById("modal-resultados");
let mensajeFinal = document.getElementById("mensaje-final");
let puntuacionFinal = document.getElementById("puntuacion-final");

if (haGanado) {
  mensajeFinal.textContent = `¡Felicitades, ${nombreJugador}! ¡Has ganado el concurso!`;
} else {
  mensajeFinal.textContent = `¡Fin del juego, ${nombreJugador}!`;
}

// Mostramos la puntuación en formato X / Y
puntuacionFinal.textContent = `${acertadas} / ${acertadas + falladas} Correctas`;

// 4. Hacemos visible la ventana emergente quitándole la clase "oculto"
modal.classList.remove("oculto");
```

Se guarda los elementos con los que trabajará

Si el valor que ha recibido haGanado es TRUE, construye el mensaje ganador

Si no es TRUE, construye el mensaje perdedor.

EN AMBOS CASOS, MUESTRA LAS POSICIONES FINALES, CON UNA CONSTRUCCION DE STRING

LA CLAVE: Para hacer visible el modal, entra el Elemento HTML almacenado en modal, y del objeto classList usa la propiedad remove de "oculto". Lo cual hará visible la caja

UN DETALLE: LA CONSTRUCCIÓN DE LOS STRINGS CON VARIABLES

```
mensajeFinal.textContent = `¡Felicitades, ${nombreJugador}! ¡Has ganado el concurso!`;
```

Al utilizar comillas invertidas, podemos incrustar variables al string envolviendolas entre `${nombreVariable}`

Esto nos evita tener que hacer esto:

"Felicitades," + nombreJugador + "! ¡Has ganado el concurso!"

Es mas limpio.